

MODULE - 3

4.4 Top-Down Parsing

Top-down parsing can be viewed as the problem of constructing a parse tree for the input string, starting from the root and creating the nodes of the parse tree in preorder (depth-first, as discussed in Section 2.3.4). Equivalently, top-down parsing can be viewed as finding a leftmost derivation for an input string.

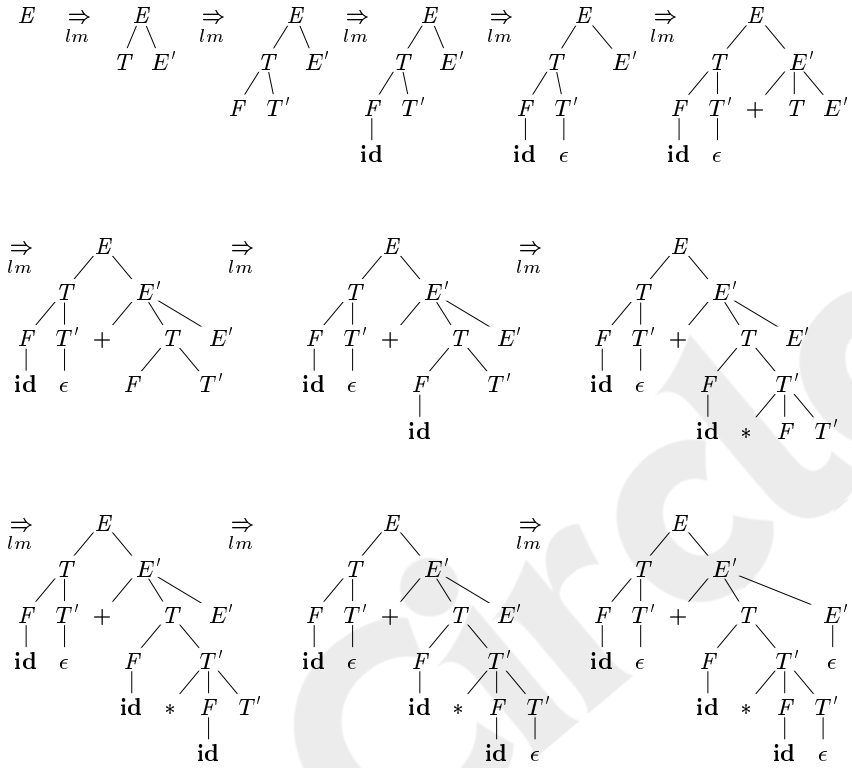
Example 4.27: The sequence of parse trees in Fig. 4.12 for the input **id+id*id** is a top-down parse according to grammar (4.2), repeated here:

$$\begin{array}{ll} E & \rightarrow T E' \\ E' & \rightarrow + T E' \mid \epsilon \\ T & \rightarrow F T' \\ T' & \rightarrow * F T' \mid \epsilon \\ F & \rightarrow (E) \mid \text{id} \end{array} \quad (4.28)$$

This sequence of trees corresponds to a leftmost derivation of the input. $\square$

At each step of a top-down parse, the key problem is that of determining the production to be applied for a nonterminal, say A . Once an A -production is chosen, the rest of the parsing process consists of “matching” the terminal symbols in the production body with the input string.

The section begins with a general form of top-down parsing, called recursive-descent parsing, which may require backtracking to find the correct A -production to be applied. Section 2.4.2 introduced predictive parsing, a special case of recursive-descent parsing, where no backtracking is required. Predictive parsing chooses the correct A -production by looking ahead at the input a fixed number of symbols, typically we may look only at one (that is, the next input symbol).

Figure 4.12: Top-down parse for $\text{id} + \text{id} * \text{id}$

For example, consider the top-down parse in Fig. 4.12, which constructs a tree with two nodes labeled E' . At the first E' node (in preorder), the production $E' \rightarrow +TE'$ is chosen; at the second E' node, the production $E' \rightarrow \epsilon$ is chosen. A predictive parser can choose between E' -productions by looking at the next input symbol.

The class of grammars for which we can construct predictive parsers looking k symbols ahead in the input is sometimes called the $LL(k)$ class. We discuss the $LL(1)$ class in Section 4.4.3, but introduce certain computations, called FIRST and FOLLOW, in a preliminary Section 4.4.2. From the FIRST and FOLLOW sets for a grammar, we shall construct “predictive parsing tables,” which make explicit the choice of production during top-down parsing. These sets are also useful during bottom-up parsing, as we shall see.

In Section 4.4.4 we give a nonrecursive parsing algorithm that maintains a stack explicitly, rather than implicitly via recursive calls. Finally, in Section 4.4.5 we discuss error recovery during top-down parsing.

4.4.1 Recursive-Descent Parsing

```

void A() {
1)      Choose an  $A$ -production,  $A \rightarrow X_1 X_2 \cdots X_k$ ;
2)      for (  $i = 1$  to  $k$  ) {
3)          if (  $X_i$  is a nonterminal )
4)              call procedure  $X_i()$ ;
5)          else if (  $X_i$  equals the current input symbol  $a$  )
6)              advance the input to the next symbol;
7)          else /* an error has occurred */;
      }
}

```

Figure 4.13: A typical procedure for a nonterminal in a top-down parser

A recursive-descent parsing program consists of a set of procedures, one for each nonterminal. Execution begins with the procedure for the start symbol, which halts and announces success if its procedure body scans the entire input string. Pseudocode for a typical nonterminal appears in Fig. 4.13. Note that this pseudocode is nondeterministic, since it begins by choosing the A -production to apply in a manner that is not specified.

General recursive-descent may require backtracking; that is, it may require repeated scans over the input. However, backtracking is rarely needed to parse programming language constructs, so backtracking parsers are not seen frequently. Even for situations like natural language parsing, backtracking is not very efficient, and tabular methods such as the dynamic programming algorithm of Exercise 4.4.9 or the method of Earley (see the bibliographic notes) are preferred.

To allow backtracking, the code of Fig. 4.13 needs to be modified. First, we cannot choose a unique A -production at line (1), so we must try each of several productions in some order. Then, failure at line (7) is not ultimate failure, but suggests only that we need to return to line (1) and try another A -production. Only if there are no more A -productions to try do we declare that an input error has been found. In order to try another A -production, we need to be able to reset the input pointer to where it was when we first reached line (1). Thus, a local variable is needed to store this input pointer for future use.

Example 4.29: Consider the grammar

$$\begin{array}{lcl}
 S & \rightarrow & c A d \\
 A & \rightarrow & a b \mid a
 \end{array}$$

To construct a parse tree top-down for the input string $w = cad$, begin with a tree consisting of a single node labeled S , and the input pointer pointing to c , the first symbol of w . S has only one production, so we use it to expand S and

obtain the tree of Fig. 4.14(a). The leftmost leaf, labeled c , matches the first symbol of input w , so we advance the input pointer to a , the second symbol of w , and consider the next leaf, labeled A .

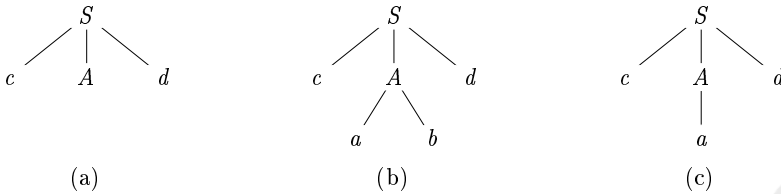


Figure 4.14: Steps in a top-down parse

Now, we expand A using the first alternative $A \rightarrow a b$ to obtain the tree of Fig. 4.14(b). We have a match for the second input symbol, a , so we advance the input pointer to d , the third input symbol, and compare d against the next leaf, labeled b . Since b does not match d , we report failure and go back to A to see whether there is another alternative for A that has not been tried, but that might produce a match.

In going back to A , we must reset the input pointer to position 2, the position it had when we first came to A , which means that the procedure for A must store the input pointer in a local variable.

The second alternative for A produces the tree of Fig. 4.14(c). The leaf a matches the second symbol of w and the leaf d matches the third symbol. Since we have produced a parse tree for w , we halt and announce successful completion of parsing. $\square$

A left-recursive grammar can cause a recursive-descent parser, even one with backtracking, to go into an infinite loop. That is, when we try to expand a nonterminal A , we may eventually find ourselves again trying to expand A without having consumed any input.

4.4.2 FIRST and FOLLOW

The construction of both top-down and bottom-up parsers is aided by two functions, FIRST and FOLLOW, associated with a grammar G . During top-down parsing, FIRST and FOLLOW allow us to choose which production to apply, based on the next input symbol. During panic-mode error recovery, sets of tokens produced by FOLLOW can be used as synchronizing tokens.

Define $FIRST(\alpha)$, where α is any string of grammar symbols, to be the set of terminals that begin strings derived from α . If $\alpha \xRightarrow{*} \epsilon$, then ϵ is also in $FIRST(\alpha)$. For example, in Fig. 4.15, $A \xRightarrow{*} c\gamma$, so c is in $FIRST(A)$.

For a preview of how FIRST can be used during predictive parsing, consider two A -productions $A \rightarrow \alpha \mid \beta$, where $FIRST(\alpha)$ and $FIRST(\beta)$ are disjoint sets. We can then choose between these A -productions by looking at the next input

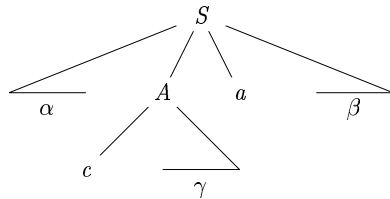


Figure 4.15: Terminal c is in $\text{FIRST}(A)$ and a is in $\text{FOLLOW}(A)$

symbol a , since a can be in at most one of $\text{FIRST}(\alpha)$ and $\text{FIRST}(\beta)$, not both. For instance, if a is in $\text{FIRST}(\beta)$ choose the production $A \rightarrow \beta$. This idea will be explored when LL(1) grammars are defined in Section 4.4.3.

Define $\text{FOLLOW}(A)$, for nonterminal A , to be the set of terminals a that can appear immediately to the right of A in some sentential form; that is, the set of terminals a such that there exists a derivation of the form $S \xRightarrow{*} \alpha A a \beta$, for some α and β , as in Fig. 4.15. Note that there may have been symbols between A and a , at some time during the derivation, but if so, they derived ϵ and disappeared. In addition, if A can be the rightmost symbol in some sentential form, then $\$$ is in $\text{FOLLOW}(A)$; recall that $\$$ is a special “endmarker” symbol that is assumed not to be a symbol of any grammar.

To compute $\text{FIRST}(X)$ for all grammar symbols X , apply the following rules until no more terminals or ϵ can be added to any FIRST set.

1. If X is a terminal, then $\text{FIRST}(X) = \{X\}$.
2. If X is a nonterminal and $X \rightarrow Y_1 Y_2 \cdots Y_k$ is a production for some $k \geq 1$, then place a in $\text{FIRST}(X)$ if for some i , a is in $\text{FIRST}(Y_i)$, and ϵ is in all of $\text{FIRST}(Y_1), \dots, \text{FIRST}(Y_{i-1})$; that is, $Y_1 \cdots Y_{i-1} \xRightarrow{*} \epsilon$. If ϵ is in $\text{FIRST}(Y_j)$ for all $j = 1, 2, \dots, k$, then add ϵ to $\text{FIRST}(X)$. For example, everything in $\text{FIRST}(Y_1)$ is surely in $\text{FIRST}(X)$. If Y_1 does not derive ϵ , then we add nothing more to $\text{FIRST}(X)$, but if $Y_1 \xRightarrow{*} \epsilon$, then we add $\text{FIRST}(Y_2)$, and so on.
3. If $X \rightarrow \epsilon$ is a production, then add ϵ to $\text{FIRST}(X)$.

Now, we can compute FIRST for any string $X_1 X_2 \cdots X_n$ as follows. Add to $\text{FIRST}(X_1 X_2 \cdots X_n)$ all non- ϵ symbols of $\text{FIRST}(X_1)$. Also add the non- ϵ symbols of $\text{FIRST}(X_2)$, if ϵ is in $\text{FIRST}(X_1)$; the non- ϵ symbols of $\text{FIRST}(X_3)$, if ϵ is in $\text{FIRST}(X_1)$ and $\text{FIRST}(X_2)$; and so on. Finally, add ϵ to $\text{FIRST}(X_1 X_2 \cdots X_n)$ if, for all i , ϵ is in $\text{FIRST}(X_i)$.

To compute $\text{FOLLOW}(A)$ for all nonterminals A , apply the following rules until nothing can be added to any FOLLOW set.

1. Place $\$$ in $\text{FOLLOW}(S)$, where S is the start symbol, and $\$$ is the input right endmarker.

2. If there is a production $A \rightarrow \alpha B \beta$, then everything in $\text{FIRST}(\beta)$ except ϵ is in $\text{FOLLOW}(B)$.
3. If there is a production $A \rightarrow \alpha B$, or a production $A \rightarrow \alpha B \beta$, where $\text{FIRST}(\beta)$ contains ϵ , then everything in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$.

Example 4.30 : Consider again the non-left-recursive grammar (4.28). Then:

1. $\text{FIRST}(F) = \text{FIRST}(T) = \text{FIRST}(E) = \{ (, \mathbf{id} \}$. To see why, note that the two productions for F have bodies that start with these two terminal symbols, $\mathbf{id}$ and the left parenthesis. T has only one production, and its body starts with F . Since F does not derive ϵ , $\text{FIRST}(T)$ must be the same as $\text{FIRST}(F)$. The same argument covers $\text{FIRST}(E)$.
2. $\text{FIRST}(E') = \{ +, \epsilon \}$. The reason is that one of the two productions for E' has a body that begins with terminal $+$, and the other's body is ϵ . Whenever a nonterminal derives ϵ , we place ϵ in FIRST for that nonterminal.
3. $\text{FIRST}(T') = \{ *, \epsilon \}$. The reasoning is analogous to that for $\text{FIRST}(E')$.
4. $\text{FOLLOW}(E) = \text{FOLLOW}(E') = \{), \$ \}$. Since E is the start symbol, $\text{FOLLOW}(E)$ must contain $\$$. The production body (E) explains why the right parenthesis is in $\text{FOLLOW}(E)$. For E' , note that this nonterminal appears only at the ends of bodies of E -productions. Thus, $\text{FOLLOW}(E')$ must be the same as $\text{FOLLOW}(E)$.
5. $\text{FOLLOW}(T) = \text{FOLLOW}(T') = \{ +,), \$ \}$. Notice that T appears in bodies only followed by E' . Thus, everything except ϵ that is in $\text{FIRST}(E')$ must be in $\text{FOLLOW}(T)$; that explains the symbol $+$. However, since $\text{FIRST}(E')$ contains ϵ (i.e., $E' \xRightarrow{*} \epsilon$), and E' is the entire string following T in the bodies of the E -productions, everything in $\text{FOLLOW}(E)$ must also be in $\text{FOLLOW}(T)$. That explains the symbols $\$$ and the right parenthesis. As for T' , since it appears only at the ends of the T -productions, it must be that $\text{FOLLOW}(T') = \text{FOLLOW}(T)$.
6. $\text{FOLLOW}(F) = \{ +, *,), \$ \}$. The reasoning is analogous to that for T in point (5).

□

4.4.3 LL(1) Grammars

Predictive parsers, that is, recursive-descent parsers needing no backtracking, can be constructed for a class of grammars called LL(1). The first “L” in LL(1) stands for scanning the input from left to right, the second “L” for producing a leftmost derivation, and the “1” for using one input symbol of lookahead at each step to make parsing action decisions.

Transition Diagrams for Predictive Parsers

Transition diagrams are useful for visualizing predictive parsers. For example, the transition diagrams for nonterminals E and E' of grammar (4.28) appear in Fig. 4.16(a). To construct the transition diagram from a grammar, first eliminate left recursion and then left factor the grammar. Then, for each nonterminal A ,

1. Create an initial and final (return) state.
2. For each production $A \rightarrow X_1 X_2 \cdots X_k$, create a path from the initial to the final state, with edges labeled $X_1, X_2, \dots, X_k$. If $A \rightarrow \epsilon$, the path is an edge labeled ϵ .

Transition diagrams for predictive parsers differ from those for lexical analyzers. Parsers have one diagram for each nonterminal. The labels of edges can be tokens or nonterminals. A transition on a token (terminal) means that we take that transition if that token is the next input symbol. A transition on a nonterminal A is a call of the procedure for A .

With an LL(1) grammar, the ambiguity of whether or not to take an ϵ -edge can be resolved by making ϵ -transitions the default choice.

Transition diagrams can be simplified, provided the sequence of grammar symbols along paths is preserved. We may also substitute the diagram for a nonterminal A in place of an edge labeled A . The diagrams in Fig. 4.16(a) and (b) are equivalent: if we trace paths from E to an accepting state and substitute for E' , then, in both sets of diagrams, the grammar symbols along the paths make up strings of the form $T + T + \cdots + T$. The diagram in (b) can be obtained from (a) by transformations akin to those in Section 2.5.4, where we used tail-recursion removal and substitution of procedure bodies to optimize the procedure for a nonterminal.

The class of LL(1) grammars is rich enough to cover most programming constructs, although care is needed in writing a suitable grammar for the source language. For example, no left-recursive or ambiguous grammar can be LL(1).

A grammar G is LL(1) if and only if whenever $A \rightarrow \alpha \mid \beta$ are two distinct productions of G , the following conditions hold:

1. For no terminal a do both α and β derive strings beginning with a .
2. At most one of α and β can derive the empty string.
3. If $\beta \xRightarrow{*} \epsilon$, then α does not derive any string beginning with a terminal in FOLLOW(A). Likewise, if $\alpha \xRightarrow{*} \epsilon$, then β does not derive any string beginning with a terminal in FOLLOW(A).

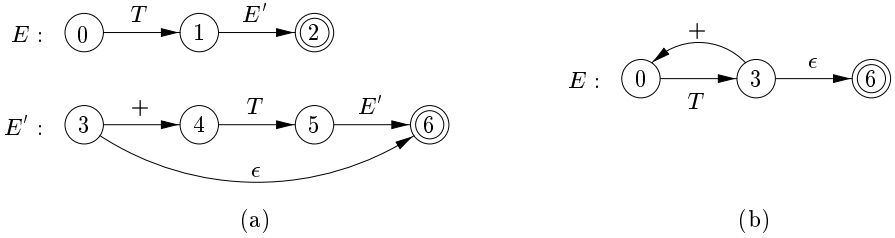


Figure 4.16: Transition diagrams for nonterminals E and E' of grammar 4.28

The first two conditions are equivalent to the statement that $\text{FIRST}(\alpha)$ and $\text{FIRST}(\beta)$ are disjoint sets. The third condition is equivalent to stating that if ϵ is in $\text{FIRST}(\beta)$, then $\text{FIRST}(\alpha)$ and $\text{FOLLOW}(A)$ are disjoint sets, and likewise if ϵ is in $\text{FIRST}(\alpha)$.

Predictive parsers can be constructed for LL(1) grammars since the proper production to apply for a nonterminal can be selected by looking only at the current input symbol. Flow-of-control constructs, with their distinguishing keywords, generally satisfy the LL(1) constraints. For instance, if we have the productions

$$\begin{array}{lcl} \text{stmt} & \rightarrow & \text{if (expr) stmt else stmt} \\ & | & \text{while (expr) stmt} \\ & | & \{ \text{stmt_list} \} \end{array}$$

then the keywords **if**, **while**, and the symbol $\{$ tell us which alternative is the only one that could possibly succeed if we are to find a statement.

The next algorithm collects the information from FIRST and FOLLOW sets into a predictive parsing table $M[A, a]$, a two-dimensional array, where A is a nonterminal, and a is a terminal or the symbol $\$,$ the input endmarker. The algorithm is based on the following idea: the production $A \rightarrow \alpha$ is chosen if the next input symbol a is in $\text{FIRST}(\alpha)$. The only complication occurs when $\alpha = \epsilon$ or, more generally, $\alpha \xRightarrow{*} \epsilon$. In this case, we should again choose $A \rightarrow \alpha$, if the current input symbol is in $\text{FOLLOW}(A)$, or if the $\$$ on the input has been reached and $\$$ is in $\text{FOLLOW}(A)$.

Algorithm 4.31: Construction of a predictive parsing table.

INPUT: Grammar G .

OUTPUT: Parsing table M .

METHOD: For each production $A \rightarrow \alpha$ of the grammar, do the following:

1. For each terminal a in $\text{FIRST}(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$.
2. If ϵ is in $\text{FIRST}(\alpha)$, then for each terminal b in $\text{FOLLOW}(A)$, add $A \rightarrow \alpha$ to $M[A, b]$. If ϵ is in $\text{FIRST}(\alpha)$ and $\$$ is in $\text{FOLLOW}(A)$, add $A \rightarrow \alpha$ to $M[A, \$]$ as well.

If, after performing the above, there is no production at all in $M[A, a]$, then set $M[A, a]$ to **error** (which we normally represent by an empty entry in the table). $\square$

Example 4.32: For the expression grammar (4.28), Algorithm 4.31 produces the parsing table in Fig. 4.17. Blanks are error entries; nonblanks indicate a production with which to expand a nonterminal.

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

Figure 4.17: Parsing table M for Example 4.32

Consider production $E \rightarrow TE'$. Since

$$\text{FIRST}(TE') = \text{FIRST}(T) = \{(\text{id})\}$$

this production is added to $M[E, (]$ and $M[E, \text{id}]$. Production $E' \rightarrow +TE'$ is added to $M[E', +]$ since $\text{FIRST}(+TE') = \{+\}$. Since $\text{FOLLOW}(E') = \{), \$\}$, production $E' \rightarrow \epsilon$ is added to $M[E',)]$ and $M[E', \$]$. $\square$

Algorithm 4.31 can be applied to any grammar G to produce a parsing table M . For every LL(1) grammar, each parsing-table entry uniquely identifies a production or signals an error. For some grammars, however, M may have some entries that are multiply defined. For example, if G is left-recursive or ambiguous, then M will have at least one multiply defined entry. Although left-recursion elimination and left factoring are easy to do, there are some grammars for which no amount of alteration will produce an LL(1) grammar.

The language in the following example has no LL(1) grammar at all.

Example 4.33: The following grammar, which abstracts the dangling-else problem, is repeated here from Example 4.22:

$$\begin{aligned} S &\rightarrow iEtSS' \mid a \\ S' &\rightarrow eS \mid \epsilon \\ E &\rightarrow b \end{aligned}$$

The parsing table for this grammar appears in Fig. 4.18. The entry for $M[S', e]$ contains both $S' \rightarrow eS$ and $S' \rightarrow \epsilon$.

The grammar is ambiguous and the ambiguity is manifested by a choice in what production to use when an e (**else**) is seen. We can resolve this ambiguity

NON - TERMINAL	INPUT SYMBOL					
	<i>a</i>	<i>b</i>	<i>e</i>	<i>i</i>	<i>t</i>	\$
<i>S</i>	$S \rightarrow a$			$S \rightarrow iEtSS'$		
<i>S'</i>			$S' \rightarrow \epsilon$ $S' \rightarrow eS$			$S' \rightarrow \epsilon$
<i>E</i>		$E \rightarrow b$				

Figure 4.18: Parsing table *M* for Example 4.33

by choosing $S' \rightarrow eS$. This choice corresponds to associating an **else** with the closest previous **then**. Note that the choice $S' \rightarrow \epsilon$ would prevent *e* from ever being put on the stack or removed from the input, and is surely wrong. $\square$

4.4.4 Nonrecursive Predictive Parsing

A nonrecursive predictive parser can be built by maintaining a stack explicitly, rather than implicitly via recursive calls. The parser mimics a leftmost derivation. If *w* is the input that has been matched so far, then the stack holds a sequence of grammar symbols α such that

$$S \xRightarrow[tm]{*} w\alpha$$

The table-driven parser in Fig. 4.19 has an input buffer, a stack containing a sequence of grammar symbols, a parsing table constructed by Algorithm 4.31, and an output stream. The input buffer contains the string to be parsed, followed by the endmarker \$. We reuse the symbol \$ to mark the bottom of the stack, which initially contains the start symbol of the grammar on top of \$.

The parser is controlled by a program that considers *X*, the symbol on top of the stack, and *a*, the current input symbol. If *X* is a nonterminal, the parser chooses an *X*-production by consulting entry $M[X, a]$ of the parsing table *M*. (Additional code could be executed here, for example, code to construct a node in a parse tree.) Otherwise, it checks for a match between the terminal *X* and current input symbol *a*.

The behavior of the parser can be described in terms of its *configurations*, which give the stack contents and the remaining input. The next algorithm describes how configurations are manipulated.

Algorithm 4.34: Table-driven predictive parsing.

INPUT: A string *w* and a parsing table *M* for grammar *G*.

OUTPUT: If *w* is in $L(G)$, a leftmost derivation of *w*; otherwise, an error indication.

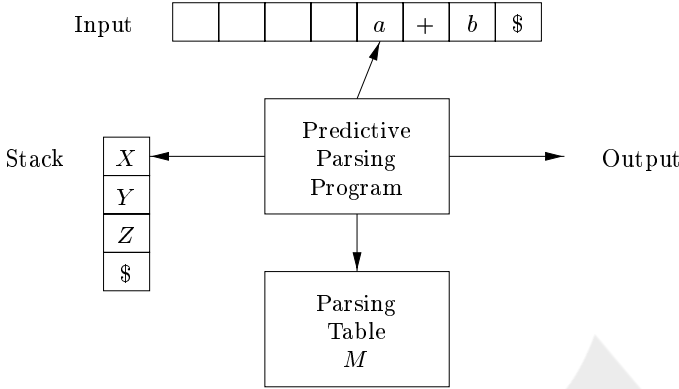


Figure 4.19: Model of a table-driven predictive parser

METHOD: Initially, the parser is in a configuration with $w\$$ in the input buffer and the start symbol S of G on top of the stack, above $\$$. The program in Fig. 4.20 uses the predictive parsing table M to produce a predictive parse for the input. $\square$

```

let  $a$  be the first symbol of  $w$ ;
let  $X$  be the top stack symbol;
while (  $X \neq \$$  ) { /* stack is not empty */
    if (  $X = a$  ) pop the stack and let  $a$  be the next symbol of  $w$ ;
    else if (  $X$  is a terminal ) error();
    else if (  $M[X, a]$  is an error entry ) error();
    else if (  $M[X, a] = X \rightarrow Y_1 Y_2 \cdots Y_k$  ) {
        output the production  $X \rightarrow Y_1 Y_2 \cdots Y_k$ ;
        pop the stack;
        push  $Y_k, Y_{k-1}, \dots, Y_1$  onto the stack, with  $Y_1$  on top;
    }
    let  $X$  be the top stack symbol;
}

```

Figure 4.20: Predictive parsing algorithm

Example 4.35 : Consider grammar (4.28); we have already seen its the parsing table in Fig. 4.17. On input **id + id * id**, the nonrecursive predictive parser of Algorithm 4.34 makes the sequence of moves in Fig. 4.21. These moves correspond to a leftmost derivation (see Fig. 4.12 for the full derivation):

$$E \xRightarrow{lm} TE' \xRightarrow{lm} FT'E' \xRightarrow{lm} \mathbf{id} T'E' \xRightarrow{lm} \mathbf{id} E' \xRightarrow{lm} \mathbf{id} + TE' \xRightarrow{lm} \cdots$$

MATCHED	STACK	INPUT	ACTION
	$E\$$	id + id * id \$	
	$TE' \$$	id + id * id \$	output $E \rightarrow TE'$
	$FT'E' \$$	id + id * id \$	output $T \rightarrow FT'$
	id $TE' \$$	id + id * id \$	output $F \rightarrow \mathbf{id}$
id	$TE' \$$	+ id * id \$	match id
id	$E' \$$	+ id * id \$	output $T' \rightarrow \epsilon$
id	+ $TE' \$$	+ id * id \$	output $E' \rightarrow + TE'$
id +	$TE' \$$	id * id \$	match +
id +	$FT'E' \$$	id * id \$	output $T \rightarrow FT'$
id +	id $TE' \$$	id * id \$	output $F \rightarrow \mathbf{id}$
id + id	$TE' \$$	* id \$	match id
id + id	* $FT'E' \$$	* id \$	output $T' \rightarrow * FT'$
id + id *	$FT'E' \$$	id \$	match *
id + id *	id $TE' \$$	id \$	output $F \rightarrow \mathbf{id}$
id + id * id	$TE' \$$	\$	match id
id + id * id	$E' \$$	\$	output $T' \rightarrow \epsilon$
id + id * id	\$	\$	output $E' \rightarrow \epsilon$

Figure 4.21: Moves made by a predictive parser on input **id + id * id**

Note that the sentential forms in this derivation correspond to the input that has already been matched (in column MATCHED) followed by the stack contents. The matched input is shown only to highlight the correspondence. For the same reason, the top of the stack is to the left; when we consider bottom-up parsing, it will be more natural to show the top of the stack to the right. The input pointer points to the leftmost symbol of the string in the INPUT column. $\square$

4.4.5 Error Recovery in Predictive Parsing

This discussion of error recovery refers to the stack of a table-driven predictive parser, since it makes explicit the terminals and nonterminals that the parser hopes to match with the remainder of the input; the techniques can also be used with recursive-descent parsing.

An error is detected during predictive parsing when the terminal on top of the stack does not match the next input symbol or when nonterminal A is on top of the stack, a is the next input symbol, and $M[A, a]$ is **error** (i.e., the parsing-table entry is empty).

Panic Mode

Panic-mode error recovery is based on the idea of skipping over symbols on the input until a token in a selected set of synchronizing tokens appears. Its

effectiveness depends on the choice of synchronizing set. The sets should be chosen so that the parser recovers quickly from errors that are likely to occur in practice. Some heuristics are as follows:

1. As a starting point, place all symbols in $\text{FOLLOW}(A)$ into the synchronizing set for nonterminal A . If we skip tokens until an element of $\text{FOLLOW}(A)$ is seen and pop A from the stack, it is likely that parsing can continue.
2. It is not enough to use $\text{FOLLOW}(A)$ as the synchronizing set for A . For example, if semicolons terminate statements, as in C, then keywords that begin statements may not appear in the FOLLOW set of the nonterminal representing expressions. A missing semicolon after an assignment may therefore result in the keyword beginning the next statement being skipped. Often, there is a hierarchical structure on constructs in a language; for example, expressions appear within statements, which appear within blocks, and so on. We can add to the synchronizing set of a lower-level construct the symbols that begin higher-level constructs. For example, we might add keywords that begin statements to the synchronizing sets for the nonterminals generating expressions.
3. If we add symbols in $\text{FIRST}(A)$ to the synchronizing set for nonterminal A , then it may be possible to resume parsing according to A if a symbol in $\text{FIRST}(A)$ appears in the input.
4. If a nonterminal can generate the empty string, then the production deriving ϵ can be used as a default. Doing so may postpone some error detection, but cannot cause an error to be missed. This approach reduces the number of nonterminals that have to be considered during error recovery.
5. If a terminal on top of the stack cannot be matched, a simple idea is to pop the terminal, issue a message saying that the terminal was inserted, and continue parsing. In effect, this approach takes the synchronizing set of a token to consist of all other tokens.

Example 4.36 : Using FIRST and FOLLOW symbols as synchronizing tokens works reasonably well when expressions are parsed according to the usual grammar (4.28). The parsing table for this grammar in Fig. 4.17 is repeated in Fig. 4.22, with “synch” indicating synchronizing tokens obtained from the FOLLOW set of the nonterminal in question. The FOLLOW sets for the nonterminals are obtained from Example 4.30.

The table in Fig. 4.22 is to be used as follows. If the parser looks up entry $M[A, a]$ and finds that it is blank, then the input symbol a is skipped. If the entry is “synch,” then the nonterminal on top of the stack is popped in an attempt to resume parsing. If a token on top of the stack does not match the input symbol, then we pop the token from the stack, as mentioned above.

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$	synch	synch
E'		$E \rightarrow +TE'$			$E \rightarrow \epsilon$	$E \rightarrow \epsilon$
T	$T \rightarrow FT'$	synch		$T \rightarrow FT'$	synch	synch
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \mathbf{id}$	synch	synch	$F \rightarrow (E)$	synch	synch

Figure 4.22: Synchronizing tokens added to the parsing table of Fig. 4.17

On the erroneous input $)\mathbf{id} * +\mathbf{id}$, the parser and error recovery mechanism of Fig. 4.22 behave as in Fig. 4.23. $\square$

STACK	INPUT	REMARK
$E \$$	$)\mathbf{id} * +\mathbf{id} \$$	error, skip $)$
$E \$$	$\mathbf{id} * +\mathbf{id} \$$	$\mathbf{id}$ is in $\text{FIRST}(E)$
$TE' \$$	$\mathbf{id} * +\mathbf{id} \$$	
$FT'E' \$$	$\mathbf{id} * +\mathbf{id} \$$	
$\mathbf{id} T'E' \$$	$\mathbf{id} * +\mathbf{id} \$$	
$T'E' \$$	$* +\mathbf{id} \$$	
$* FT'E' \$$	$* +\mathbf{id} \$$	
$FT'E' \$$	$+\mathbf{id} \$$	error, $M[F, +] = \text{synch}$
$T'E' \$$	$+\mathbf{id} \$$	F has been popped
$E' \$$	$+\mathbf{id} \$$	
$+ TE' \$$	$+\mathbf{id} \$$	
$TE' \$$	$\mathbf{id} \$$	
$FT'E' \$$	$\mathbf{id} \$$	
$\mathbf{id} T'E' \$$	$\mathbf{id} \$$	
$T'E' \$$	$\$$	
$E' \$$	$\$$	
$\$$	$\$$	

Figure 4.23: Parsing and error recovery moves made by a predictive parser

The above discussion of panic-mode recovery does not address the important issue of error messages. The compiler designer must supply informative error messages that not only describe the error, they must draw attention to where the error was discovered.

Phrase-level Recovery

Phrase-level error recovery is implemented by filling in the blank entries in the predictive parsing table with pointers to error routines. These routines may change, insert, or delete symbols on the input and issue appropriate error messages. They may also pop from the stack. Alteration of stack symbols or the pushing of new symbols onto the stack is questionable for several reasons. First, the steps carried out by the parser might then not correspond to the derivation of any word in the language at all. Second, we must ensure that there is no possibility of an infinite loop. Checking that any recovery action eventually results in an input symbol being consumed (or the stack being shortened if the end of the input has been reached) is a good way to protect against such loops.

4.4.6 Exercises for Section 4.4

Exercise 4.4.1: For each of the following grammars, devise predictive parsers and show the parsing tables. You may left-factor and/or eliminate left-recursion from your grammars first.

- a) The grammar of Exercise 4.2.2(a).
- b) The grammar of Exercise 4.2.2(b).
- c) The grammar of Exercise 4.2.2(c).
- d) The grammar of Exercise 4.2.2(d).
- e) The grammar of Exercise 4.2.2(e).
- f) The grammar of Exercise 4.2.2(g).

!! Exercise 4.4.2: Is it possible, by modifying the grammar in any way, to construct a predictive parser for the language of Exercise 4.2.1 (postfix expressions with operand a)?

Exercise 4.4.3: Compute FIRST and FOLLOW for the grammar of Exercise 4.2.1.

Exercise 4.4.4: Compute FIRST and FOLLOW for each of the grammars of Exercise 4.2.2.

Exercise 4.4.5: The grammar $S \rightarrow a S a \mid a a$ generates all even-length strings of a 's. We can devise a recursive-descent parser with backtrack for this grammar. If we choose to expand by production $S \rightarrow a a$ first, then we shall only recognize the string aa . Thus, any reasonable recursive-descent parser will try $S \rightarrow a S a$ first.

- a) Show that this recursive-descent parser recognizes inputs aa , $aaaa$, and $aaaaaaaa$, but not $aaaaaa$.

!! b) What language does this recursive-descent parser recognize?

The following exercises are useful steps in the construction of a “Chomsky Normal Form” grammar from arbitrary grammars, as defined in Exercise 4.4.8.

! Exercise 4.4.6: A grammar is ϵ -free if no production body is ϵ (called an ϵ -production).

- a) Give an algorithm to convert any grammar into an ϵ -free grammar that generates the same language (with the possible exception of the empty string — no ϵ -free grammar can generate ϵ). *Hint:* First find all the nonterminals that are *nullable*, meaning that they generate ϵ , perhaps by a long derivation.
- b) Apply your algorithm to the grammar $S \rightarrow aSbS \mid bSaS \mid \epsilon$.

! Exercise 4.4.7: A *single production* is a production whose body is a single nonterminal, i.e., a production of the form $A \rightarrow B$.

- a) Give an algorithm to convert any grammar into an ϵ -free grammar, with no single productions, that generates the same language (with the possible exception of the empty string) *Hint:* First eliminate ϵ -productions, and then find for which pairs of nonterminals A and B does $A \xRightarrow{*} B$ by a sequence of single productions.
- b) Apply your algorithm to the grammar (4.1) in Section 4.1.2.
- c) Show that, as a consequence of part (a), we can convert a grammar into an equivalent grammar that has no *cycles* (derivations of one or more steps in which $A \xRightarrow{*} A$ for some nonterminal A).

!! Exercise 4.4.8: A grammar is said to be in *Chomsky Normal Form* (CNF) if every production is either of the form $A \rightarrow BC$ or of the form $A \rightarrow a$, where A , B , and C are nonterminals, and a is a terminal. Show how to convert any grammar into a CNF grammar for the same language (with the possible exception of the empty string — no CNF grammar can generate ϵ).

! Exercise 4.4.9: Every language that has a context-free grammar can be recognized in at most $O(n^3)$ time for strings of length n . A simple way to do so, called the *Cocke-Younger-Kasami* (or CYK) algorithm is based on dynamic programming. That is, given a string $a_1a_2 \cdots a_n$, we construct an n -by- n table T such that T_{ij} is the set of nonterminals that generate the substring $a_i a_{i+1} \cdots a_j$. If the underlying grammar is in CNF (see Exercise 4.4.8), then one table entry can be filled in in $O(n)$ time, provided we fill the entries in the proper order: lowest value of $j - i$ first. Write an algorithm that correctly fills in the entries of the table, and show that your algorithm takes $O(n^3)$ time. Having filled in the table, how do you determine whether $a_1a_2 \cdots a_n$ is in the language?

! Exercise 4.4.10: Show how, having filled in the table as in Exercise 4.4.9, we can in $O(n)$ time recover a parse tree for $a_1a_2 \cdots a_n$. *Hint:* modify the table so it records, for each nonterminal A in each table entry T_{ij} , some pair of nonterminals in other table entries that justified putting A in T_{ij} .

! Exercise 4.4.11: Modify your algorithm of Exercise 4.4.9 so that it will find, for any string, the smallest number of insert, delete, and mutate errors (each error a single character) needed to turn the string into a string in the language of the underlying grammar.

$stmt$	$\rightarrow$	if e then $stmt\ stmtTail$
	$ $	while e do $stmt$
	$ $	begin $list$ end
	$ $	s
$stmtTail$	$\rightarrow$	else $stmt$
	$ $	ϵ
$list$	$\rightarrow$	$stmt\ listTail$
$listTail$	$\rightarrow$	; $list$
	$ $	ϵ

Figure 4.24: A grammar for certain kinds of statements

! Exercise 4.4.12: In Fig. 4.24 is a grammar for certain statements. You may take e and s to be terminals standing for conditional expressions and “other statements,” respectively. If we resolve the conflict regarding expansion of the optional “else” (nonterminal $stmtTail$) by preferring to consume an **else** from the input whenever we see one, we can build a predictive parser for this grammar. Using the idea of synchronizing symbols described in Section 4.4.5:

- a) Build an error-correcting predictive parsing table for the grammar.
- b) Show the behavior of your parser on the following inputs:

- (i) **if** e **then** s **;** **if** e **then** s **end**
- (ii) **while** e **do** **begin** s **;** **if** e **then** s **;** **end**

4.5 Bottom-Up Parsing

A bottom-up parse corresponds to the construction of a parse tree for an input string beginning at the leaves (the bottom) and working up towards the root (the top). It is convenient to describe parsing as the process of building parse trees, although a front end may in fact carry out a translation directly without building an explicit tree. The sequence of tree snapshots in Fig. 4.25 illustrates

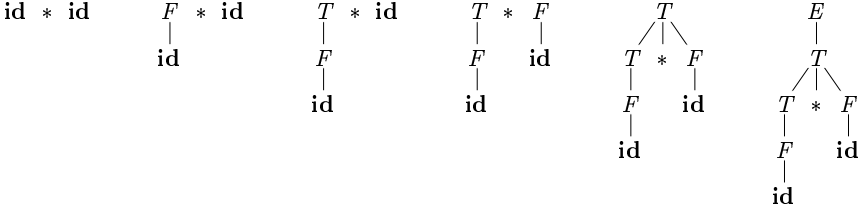


Figure 4.25: A bottom-up parse for $\text{id} * \text{id}$

a bottom-up parse of the token stream $\text{id} * \text{id}$, with respect to the expression grammar (4.1).

This section introduces a general style of bottom-up parsing known as shift-reduce parsing. The largest class of grammars for which shift-reduce parsers can be built, the LR grammars, will be discussed in Sections 4.6 and 4.7. Although it is too much work to build an LR parser by hand, tools called automatic parser generators make it easy to construct efficient LR parsers from suitable grammars. The concepts in this section are helpful for writing suitable grammars to make effective use of an LR parser generator. Algorithms for implementing parser generators appear in Section 4.7.

4.5.1 Reductions

We can think of bottom-up parsing as the process of “reducing” a string w to the start symbol of the grammar. At each *reduction* step, a specific substring matching the body of a production is replaced by the nonterminal at the head of that production.

The key decisions during bottom-up parsing are about when to reduce and about what production to apply, as the parse proceeds.

Example 4.37: The snapshots in Fig. 4.25 illustrate a sequence of reductions; the grammar is the expression grammar (4.1). The reductions will be discussed in terms of the sequence of strings

$$\text{id} * \text{id}, \quad F * \text{id}, \quad T * \text{id}, \quad T * F, \quad T, \quad E$$

The strings in this sequence are formed from the roots of all the subtrees in the snapshots. The sequence starts with the input string $\text{id} * \text{id}$. The first reduction produces $F * \text{id}$ by reducing the leftmost id to F , using the production $F \rightarrow \text{id}$. The second reduction produces $T * \text{id}$ by reducing F to T .

Now, we have a choice between reducing the string T , which is the body of $E \rightarrow T$, and the string consisting of the second id , which is the body of $F \rightarrow \text{id}$. Rather than reduce T to E , the second id is reduced to F , resulting in the string $T * F$. This string then reduces to T . The parse completes with the reduction of T to the start symbol E . $\square$

By definition, a reduction is the reverse of a step in a derivation (recall that in a derivation, a nonterminal in a sentential form is replaced by the body of one of its productions). The goal of bottom-up parsing is therefore to construct a derivation in reverse. The following corresponds to the parse in Fig. 4.25:

$$E \Rightarrow T \Rightarrow T * F \Rightarrow T * \mathbf{id} \Rightarrow F * \mathbf{id} \Rightarrow \mathbf{id} * \mathbf{id}$$

This derivation is in fact a rightmost derivation.

4.5.2 Handle Pruning

Bottom-up parsing during a left-to-right scan of the input constructs a rightmost derivation in reverse. Informally, a “handle” is a substring that matches the body of a production, and whose reduction represents one step along the reverse of a rightmost derivation.

For example, adding subscripts to the tokens **id** for clarity, the handles during the parse of **id**₁ * **id**₂ according to the expression grammar (4.1) are as in Fig. 4.26. Although *T* is the body of the production *E* → *T*, the symbol *T* is not a handle in the sentential form *T* * **id**₂. If *T* were indeed replaced by *E*, we would get the string *E* * **id**₂, which cannot be derived from the start symbol *E*. Thus, the leftmost substring that matches the body of some production need not be a handle.

RIGHT SENTENTIAL FORM	HANDLE	REDUCING PRODUCTION
id ₁ * id ₂	id ₁	<i>F</i> → id
<i>F</i> * id ₂	<i>F</i>	<i>T</i> → <i>F</i>
<i>T</i> * id ₂	id ₂	<i>F</i> → id
<i>T</i> * <i>F</i>	<i>T</i> * <i>F</i>	<i>T</i> → <i>T</i> * <i>F</i>
<i>T</i>	<i>T</i>	<i>E</i> → <i>T</i>

Figure 4.26: Handles during a parse of **id**₁ * **id**₂

Formally, if $S \overset{*}{\underset{rm}{\Rightarrow}} \alpha Aw \overset{*}{\underset{rm}{\Rightarrow}} \alpha \beta w$, as in Fig. 4.27, then production *A* → *β* in the position following *α* is a *handle* of *αβw*. Alternatively, a handle of a right-sentential form *γ* is a production *A* → *β* and a position of *γ* where the string *β* may be found, such that replacing *β* at that position by *A* produces the previous right-sentential form in a rightmost derivation of *γ*.

Notice that the string *w* to the right of the handle must contain only terminal symbols. For convenience, we refer to the body *β* rather than *A* → *β* as a handle. Note we say “a handle” rather than “the handle,” because the grammar could be ambiguous, with more than one rightmost derivation of *αβw*. If a grammar is unambiguous, then every right-sentential form of the grammar has exactly one handle.

A rightmost derivation in reverse can be obtained by “handle pruning.” That is, we start with a string of terminals *w* to be parsed. If *w* is a sentence

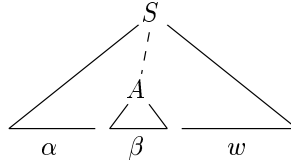


Figure 4.27: A handle $A \rightarrow \beta$ in the parse tree for $\alpha\beta w$

of the grammar at hand, then let $w = \gamma_n$, where γ_n is the n th right-sentential form of some as yet unknown rightmost derivation

$$S = \gamma_0 \xRightarrow{rm} \gamma_1 \xRightarrow{rm} \gamma_2 \xRightarrow{rm} \cdots \xRightarrow{rm} \gamma_{n-1} \xRightarrow{rm} \gamma_n = w$$

To reconstruct this derivation in reverse order, we locate the handle β_n in γ_n and replace β_n by the head of the relevant production $A_n \rightarrow \beta_n$ to obtain the previous right-sentential form γ_{n-1} . Note that we do not yet know how handles are to be found, but we shall see methods of doing so shortly.

We then repeat this process. That is, we locate the handle β_{n-1} in γ_{n-1} and reduce this handle to obtain the right-sentential form γ_{n-2} . If by continuing this process we produce a right-sentential form consisting only of the start symbol S , then we halt and announce successful completion of parsing. The reverse of the sequence of productions used in the reductions is a rightmost derivation for the input string.

4.5.3 Shift-Reduce Parsing

Shift-reduce parsing is a form of bottom-up parsing in which a stack holds grammar symbols and an input buffer holds the rest of the string to be parsed. As we shall see, the handle always appears at the top of the stack just before it is identified as the handle.

We use $\$$ to mark the bottom of the stack and also the right end of the input. Conventionally, when discussing bottom-up parsing, we show the top of the stack on the right, rather than on the left as we did for top-down parsing. Initially, the stack is empty, and the string w is on the input, as follows:

STACK	INPUT
$\$$	$w \$$

During a left-to-right scan of the input string, the parser shifts zero or more input symbols onto the stack, until it is ready to reduce a string β of grammar symbols on top of the stack. It then reduces β to the head of the appropriate production. The parser repeats this cycle until it has detected an error or until the stack contains the start symbol and the input is empty:

STACK	INPUT
\$ S	\$

Upon entering this configuration, the parser halts and announces successful completion of parsing. Figure 4.28 steps through the actions a shift-reduce parser might take in parsing the input string $\mathbf{id}_1 * \mathbf{id}_2$ according to the expression grammar (4.1).

STACK	INPUT	ACTION
\$	$\mathbf{id}_1 * \mathbf{id}_2$ \$	shift
\$ $\mathbf{id}_1$	* $\mathbf{id}_2$ \$	reduce by $F \rightarrow \mathbf{id}$
\$ F	* $\mathbf{id}_2$ \$	reduce by $T \rightarrow F$
\$ T	* $\mathbf{id}_2$ \$	shift
\$ $T *$	$\mathbf{id}_2$ \$	shift
\$ $T * \mathbf{id}_2$	\$	reduce by $F \rightarrow \mathbf{id}$
\$ $T * F$	\$	reduce by $T \rightarrow T * F$
\$ T	\$	reduce by $E \rightarrow T$
\$ E	\$	accept

Figure 4.28: Configurations of a shift-reduce parser on input $\mathbf{id}_1 * \mathbf{id}_2$

While the primary operations are shift and reduce, there are actually four possible actions a shift-reduce parser can make: (1) shift, (2) reduce, (3) accept, and (4) error.

1. *Shift*. Shift the next input symbol onto the top of the stack.
2. *Reduce*. The right end of the string to be reduced must be at the top of the stack. Locate the left end of the string within the stack and decide with what nonterminal to replace the string.
3. *Accept*. Announce successful completion of parsing.
4. *Error*. Discover a syntax error and call an error recovery routine.

The use of a stack in shift-reduce parsing is justified by an important fact: the handle will always eventually appear on top of the stack, never inside. This fact can be shown by considering the possible forms of two successive steps in any rightmost derivation. Figure 4.29 illustrates the two possible cases. In case (1), A is replaced by $\beta B y$, and then the rightmost nonterminal B in the body $\beta B y$ is replaced by γ . In case (2), A is again expanded first, but this time the body is a string y of terminals only. The next rightmost nonterminal B will be somewhere to the left of y .

In other words:

$$\begin{aligned}
 (1) \quad & S \xRightarrow{*}_{rm} \alpha A z \Rightarrow \alpha \beta B y z \xRightarrow{rm} \alpha \beta \gamma y z \\
 (2) \quad & S \xRightarrow{rm} \alpha B x A z \Rightarrow \alpha B x y z \xRightarrow{rm} \alpha \gamma x y z
 \end{aligned}$$

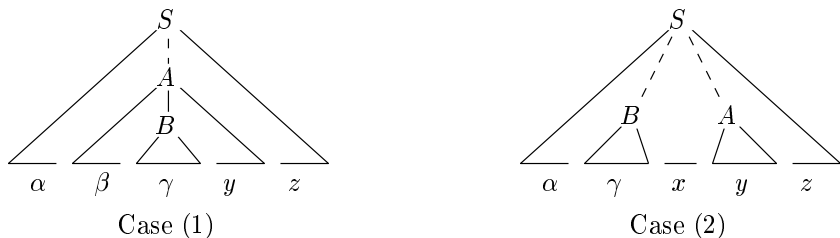


Figure 4.29: Cases for two successive steps of a rightmost derivation

Consider case (1) in reverse, where a shift-reduce parser has just reached the configuration

STACK	INPUT
$\$ \alpha \beta \gamma$	$y z \$$

The parser reduces the handle γ to B to reach the configuration

$\$ \alpha \beta B$	$y z \$$
---------------------	----------

The parser can now shift the string y onto the stack by a sequence of zero or more shift moves to reach the configuration

$\$ \alpha \beta B y$	$z \$$
-----------------------	--------

with the handle $\beta B y$ on top of the stack, and it gets reduced to A .

Now consider case (2). In configuration

$\$ \alpha \gamma$	$x y z \$$
--------------------	------------

the handle γ is on top of the stack. After reducing the handle γ to B , the parser can shift the string xy to get the next handle y on top of the stack, ready to be reduced to A :

$\$ \alpha B x y$	$z \$$
-------------------	--------

In both cases, after making a reduction the parser had to shift zero or more symbols to get the next handle onto the stack. It never had to go into the stack to find the handle.

4.5.4 Conflicts During Shift-Reduce Parsing

There are context-free grammars for which shift-reduce parsing cannot be used. Every shift-reduce parser for such a grammar can reach a configuration in which the parser, knowing the entire stack and also the next k input symbols, cannot decide whether to shift or to reduce (a *shift/reduce conflict*), or cannot decide

which of several reductions to make (a *reduce/reduce conflict*). We now give some examples of syntactic constructs that give rise to such grammars. Technically, these grammars are not in the $LR(k)$ class of grammars defined in Section 4.7; we refer to them as non-LR grammars. The k in $LR(k)$ refers to the number of symbols of lookahead on the input. Grammars used in compiling usually fall in the $LR(1)$ class, with one symbol of lookahead at most.

Example 4.38: An ambiguous grammar can never be LR. For example, consider the dangling-else grammar (4.14) of Section 4.3:

$$\begin{array}{lcl} stmt & \rightarrow & \text{if } expr \text{ then } stmt \\ & & | \text{if } expr \text{ then } stmt \text{ else } stmt \\ & & | \text{other} \end{array}$$

If we have a shift-reduce parser in configuration

STACK	INPUT
... if expr then stmt	else ... \$

we cannot tell whether **if expr then stmt** is the handle, no matter what appears below it on the stack. Here there is a shift/reduce conflict. Depending on what follows the **else** on the input, it might be correct to reduce **if expr then stmt** to *stmt*, or it might be correct to shift **else** and then to look for another *stmt* to complete the alternative **if expr then stmt else stmt**.

Note that shift-reduce parsing can be adapted to parse certain ambiguous grammars, such as the if-then-else grammar above. If we resolve the shift/reduce conflict on **else** in favor of shifting, the parser will behave as we expect, associating each **else** with the previous unmatched **then**. We discuss parsers for such ambiguous grammars in Section 4.8. $\square$

Another common setting for conflicts occurs when we know we have a handle, but the stack contents and the next input symbol are insufficient to determine which production should be used in a reduction. The next example illustrates this situation.

Example 4.39: Suppose we have a lexical analyzer that returns the token name **id** for all names, regardless of their type. Suppose also that our language invokes procedures by giving their names, with parameters surrounded by parentheses, and that arrays are referenced by the same syntax. Since the translation of indices in array references and parameters in procedure calls are different, we want to use different productions to generate lists of actual parameters and indices. Our grammar might therefore have (among others) productions such as those in Fig. 4.30.

A statement beginning with $p(i,j)$ would appear as the token stream **id(id,id)** to the parser. After shifting the first three tokens onto the stack, a shift-reduce parser would be in configuration

(1)	<i>stmt</i>	$\rightarrow$	id (<i>parameter_list</i>)
(2)	<i>stmt</i>	$\rightarrow$	<i>expr</i> := <i>expr</i>
(3)	<i>parameter_list</i>	$\rightarrow$	<i>parameter_list</i> , <i>parameter</i>
(4)	<i>parameter_list</i>	$\rightarrow$	<i>parameter</i>
(5)	<i>parameter</i>	$\rightarrow$	id
(6)	<i>expr</i>	$\rightarrow$	id (<i>expr_list</i>)
(7)	<i>expr</i>	$\rightarrow$	id
(8)	<i>expr_list</i>	$\rightarrow$	<i>expr_list</i> , <i>expr</i>
(9)	<i>expr_list</i>	$\rightarrow$	<i>expr</i>

Figure 4.30: Productions involving procedure calls and array references

STACK	INPUT
... id (id	, id) ...

It is evident that the **id** on top of the stack must be reduced, but by which production? The correct choice is production (5) if *p* is a procedure, but production (7) if *p* is an array. The stack does not tell which; information in the symbol table obtained from the declaration of *p* must be used.

One solution is to change the token **id** in production (1) to **procid** and to use a more sophisticated lexical analyzer that returns the token name **procid** when it recognizes a lexeme that is the name of a procedure. Doing so would require the lexical analyzer to consult the symbol table before returning a token.

If we made this modification, then on processing *p*(*i*,*j*) the parser would be either in the configuration

STACK	INPUT
... procid (id	, id) ...

or in the configuration above. In the former case, we choose reduction by production (5); in the latter case by production (7). Notice how the symbol third from the top of the stack determines the reduction to be made, even though it is not involved in the reduction. Shift-reduce parsing can utilize information far down in the stack to guide the parse. $\square$

4.5.5 Exercises for Section 4.5

Exercise 4.5.1: For the grammar $S \rightarrow 0S1 \mid 01$ of Exercise 4.2.2(a), indicate the handle in each of the following right-sentential forms:

a) 000111.

b) 00S11.

Exercise 4.5.2: Repeat Exercise 4.5.1 for the grammar $S \rightarrow SS+ \mid SS* \mid a$ of Exercise 4.2.1 and the following right-sentential forms:

- a) $SSS + a * +$.
- b) $SS + a * a +$.
- c) $aaa * a + +$.

Exercise 4.5.3: Give bottom-up parses for the following input strings and grammars:

- a) The input 000111 according to the grammar of Exercise 4.5.1.
- b) The input $aaa * a + +$ according to the grammar of Exercise 4.5.2.